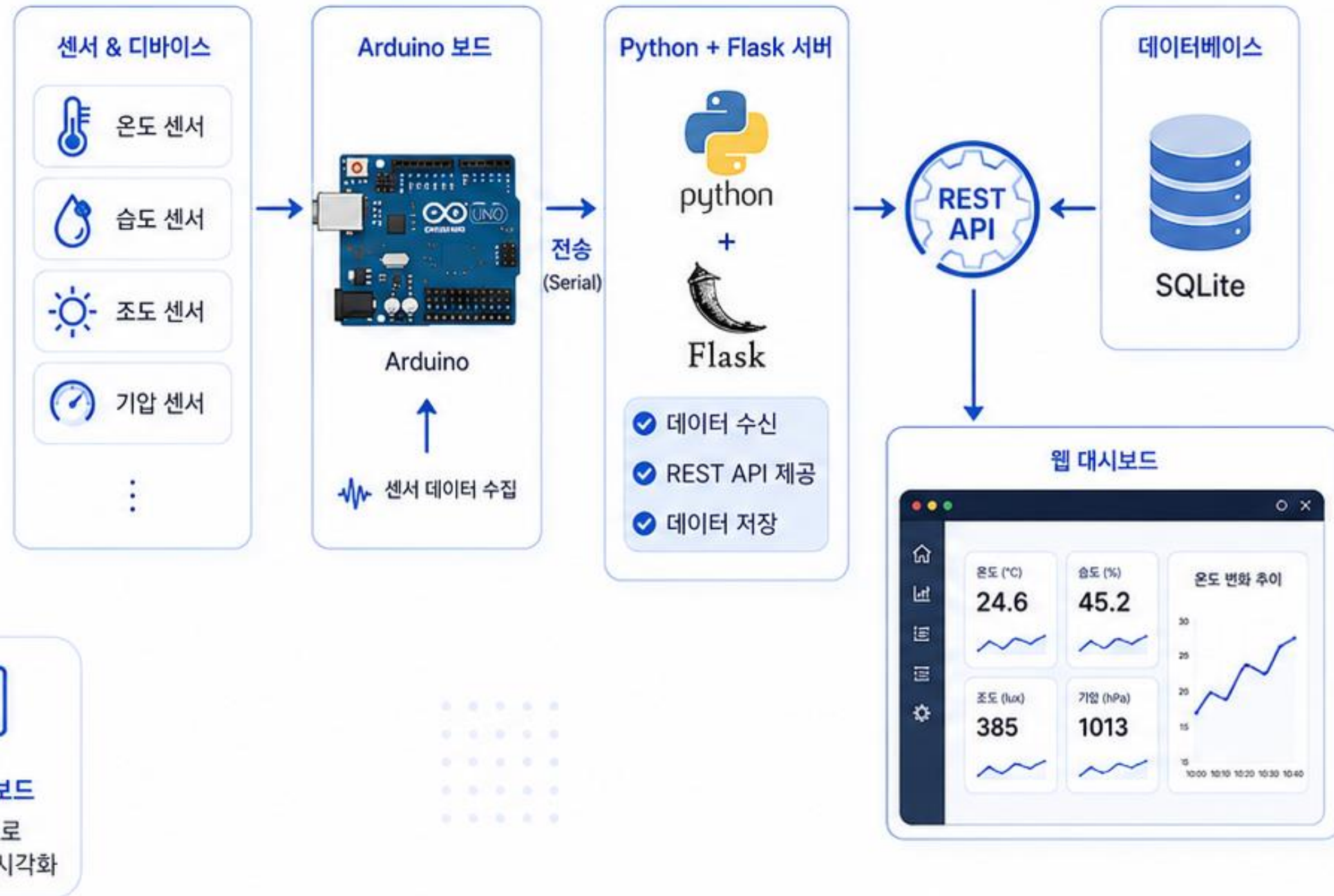


Day 4. Flask · REST API · IoT Dashboard

센서 데이터를 웹 서버와 대시보드로 연결하기

Python 기반 IoT 모니터링 서버 구축



웹 서버 구축
Flask 기반
IoT 서버 구현



REST API 제공
센서 데이터
JSON API 제공



데이터 저장
SQLite를 활용한
안정적인 데이터 관리



실시간 대시보드
웹 대시보드로
실시간 데이터 시각화

오늘의 학습 목표

오늘은 센서 데이터를 웹 서비스로 확장하는 전체 흐름을 학습합니다.



1 웹 서버와 HTTP 구조 이해



2 Flask 기본 서버 만들기



3 REST API로 센서 데이터 제공

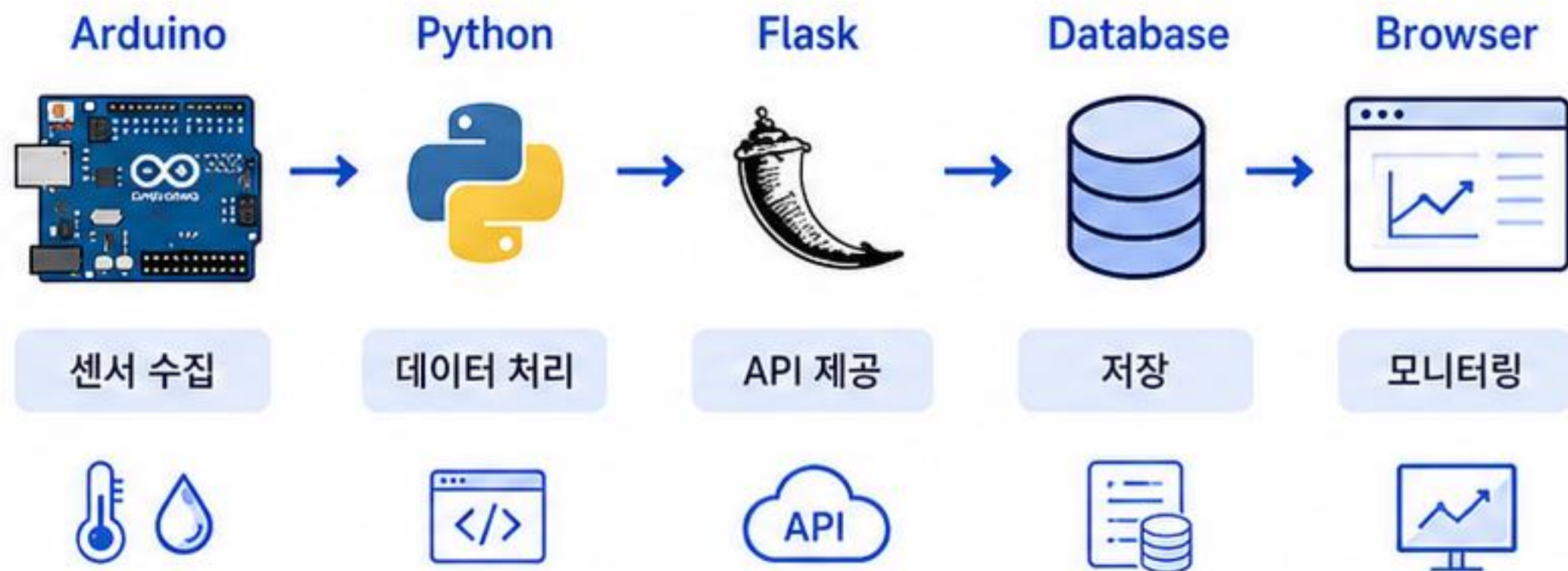


4 SQLite에 센서 로그 저장



5 웹 대시보드로 실시간 모니터링

오늘의 학습 흐름



핵심 흐름 : 센서 데이터 수집 → 웹 서버 제공 → DB 저장 → 대시보드 시각화

Day 3 핵심 복습

객체지향부터 데이터 처리까지, 오늘은 이 흐름을 웹 서비스로 확장합니다.

✓  클래스와 객체

✓  Sensor 클래스 설계

✓  dataclass 활용

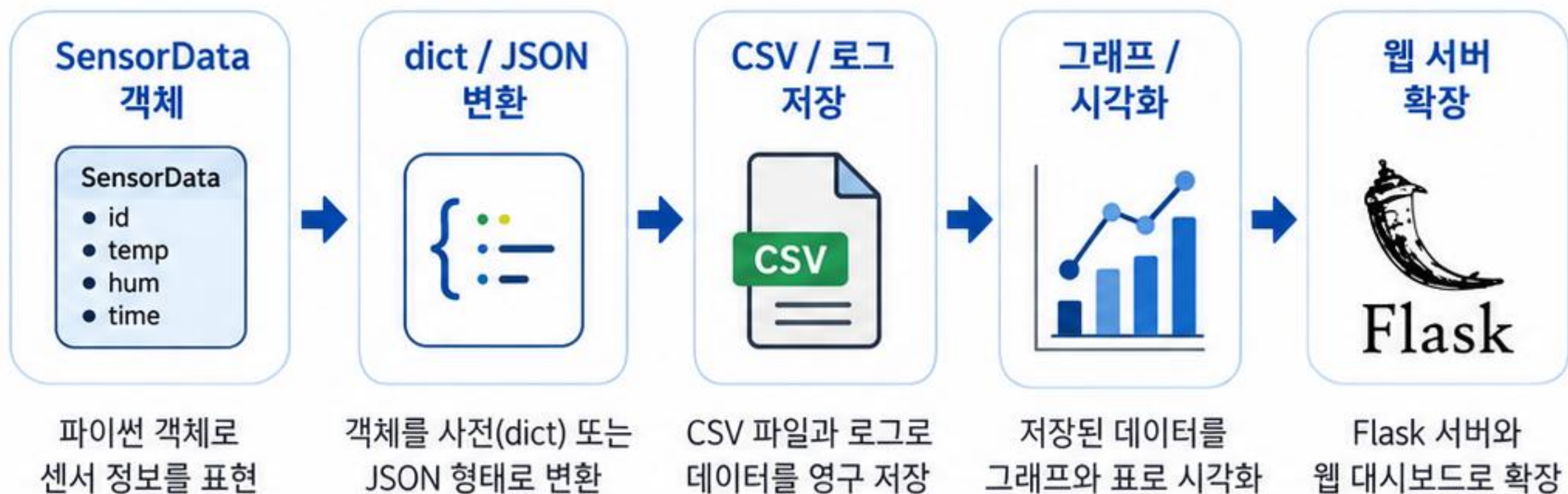
✓  예외 처리

✓  로깅과 실행 기록

✓  데이터 검증

✓  분석과 시각화

Day 3 지식이 Day 4로 이어지는 흐름



오늘은 저장된 데이터를 Flask 서버와 웹 대시보드에서 확인할 수 있도록 확장합니다.

왜 IoT 시스템에 웹 서버가 필요한가?

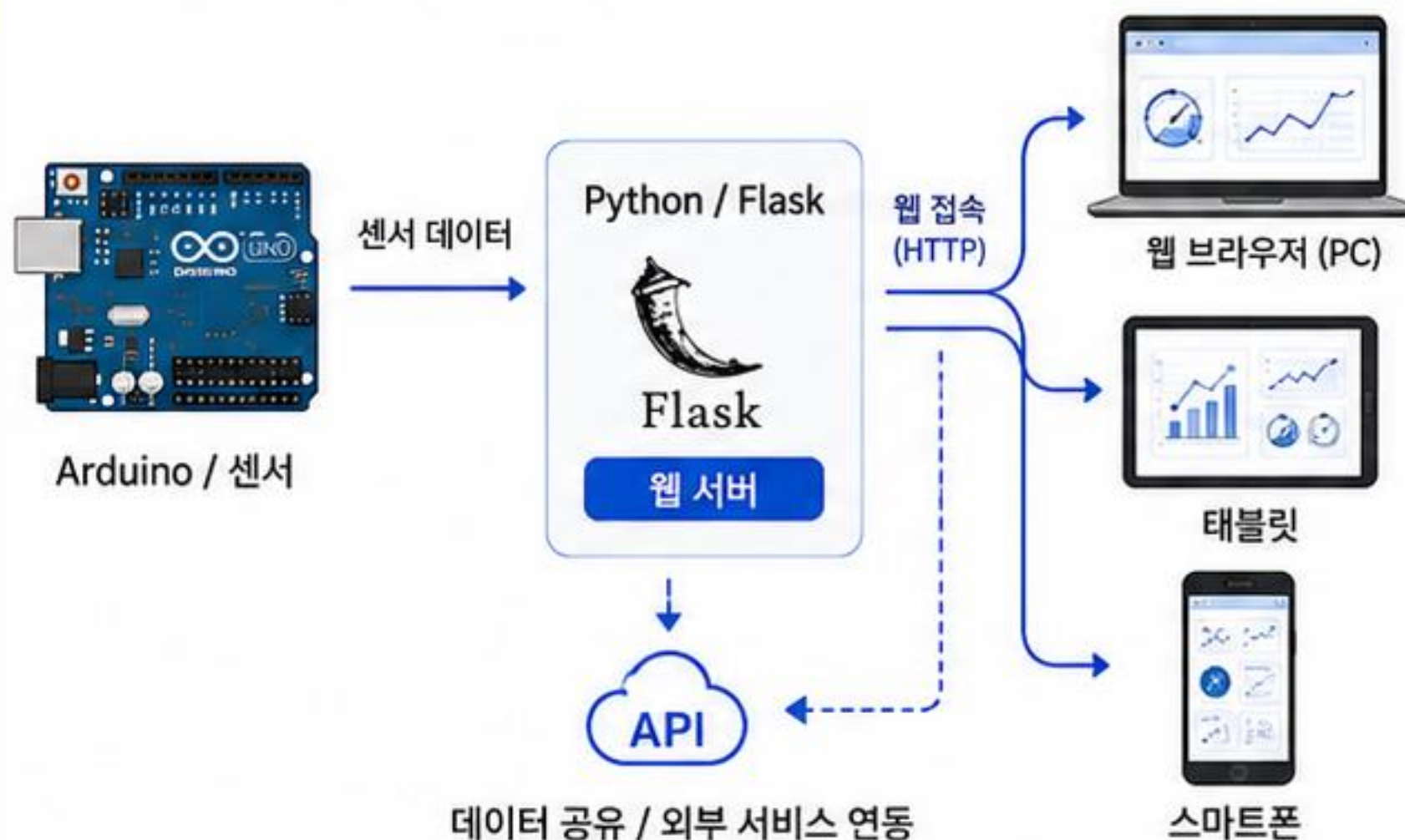
센서 데이터를 여러 사용자와 서비스가 활용할 수 있도록 확장합니다.

기존 방식 (로컬 확인만 가능)



- × 하나의 PC에서만 확인 가능
- × 원격 접근이 어렵고 불편함
- × 데이터 공유 및 서비스 활용이 어려움

웹 서버 확장 (다양한 사용자와 서비스로 확장)



- 원격 모니터링
언제 어디서나 확인
- 여러 사용자가 접근 가능
동시 접속 지원
- 데이터 공유
저장, 분석, 백업 용이
- API 기반 연동
다른 서비스와 연동 가능
- 대시보드 제공
시각화로 빠른 이해



센서 데이터는 웹 서버를 통해 서비스가 됩니다. 이제 여러분의 IoT 시스템을 더 넓은 세상과 연결해 보세요!

웹은 어떻게 동작하는가?

브라우저의 요청(Request)과 서버의 응답(Response)으로 웹 서비스가 동작합니다.



핵심
개념

1 클라이언트가 요청을 보낸다 → 2 서버가 요청을 처리한다 → 3 응답 데이터를 만든다 → 4 브라우저가 화면에 보여준다



IoT에서는 센서 데이터를 API 또는 웹 페이지 형태로 제공할 수 있습니다.

웹에서 데이터를 주고받을 때는 URL과 HTTP 메서드를 사용합니다.
API는 URL(경로)과 HTTP 메서드로 기능을 구분합니다.



Point

URL은 자원의 위치를 나타내고,
HTTP 메서드는 그 자원에 대해 수행할 작업을 정의합니다.

1. URL 구조

`http://localhost:5000/sensor/latest`

프로토콜

http 또는 https
통신 규약(규칙)

호스트(도메인)

서버 주소
(예: localhost,
192.168.0.10)

포트

서버가 열려 있는
통신 포트
(기본 80 또는 443)

경로(Path)

서버 내 자원의 위치
(API 엔드포인트)

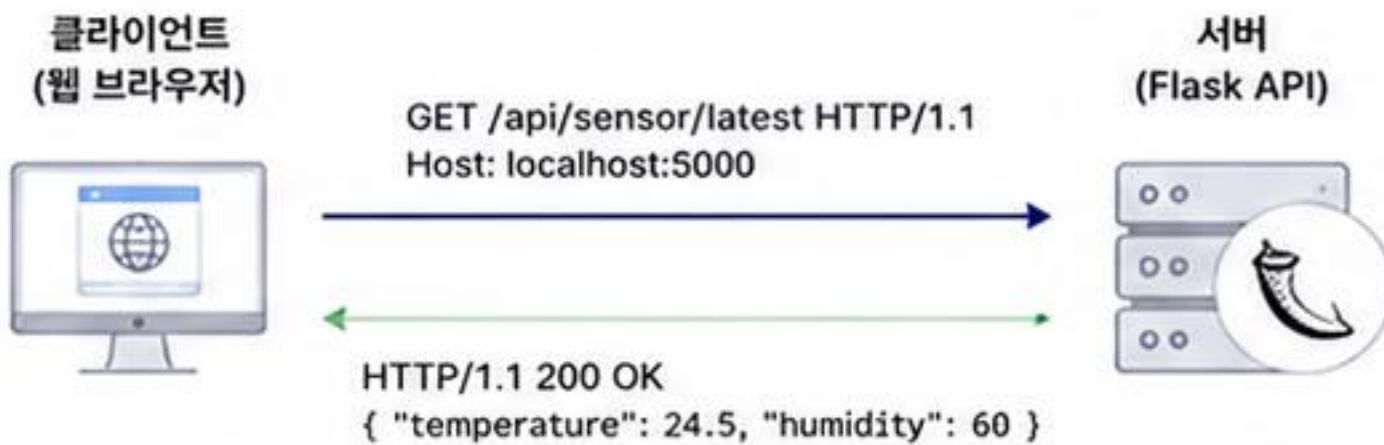
★ 예시 URL 해석

`http://localhost:5000/sensor/latest` → 로컬 서버(5000번 포트)의 sensor 최신 데이터를 요청

2. HTTP 메서드 종류

메서드	설명	특징	IoT/센서 예시
GET	데이터 조회	서버의 데이터를 조회할 때 사용	센서 최신값 조회 (GET /api/sensor/latest)
POST	데이터 생성	새로운 데이터를 서버에 등록할 때 사용	센서 데이터 등록 (POST /api/sensor)
PUT	데이터 수정(전체)	기존 데이터를 덮어쓸 때 사용	설정값 변경 (PUT /api/config)
DELETE	데이터 삭제	지정한 데이터를 삭제할 때 사용	특정 로그 삭제 (DELETE /api/logs/10)

3. HTTP 요청 예시 (GET)



4. URL(경로) vs 메서드(행동)

같은 URL이라도 메서드에 따라 기능이 달라집니다.

예시: /api/sensor		
GET	/api/sensor	→ 센서 목록 조회
POST	/api/sensor	→ 새 센서 데이터 등록
PUT	/api/sensor/1	→ ID=1 센서 데이터 수정
DELETE	/api/sensor/1	→ ID=1 센서 데이터 삭제

5. IoT API에서의 활용 예시



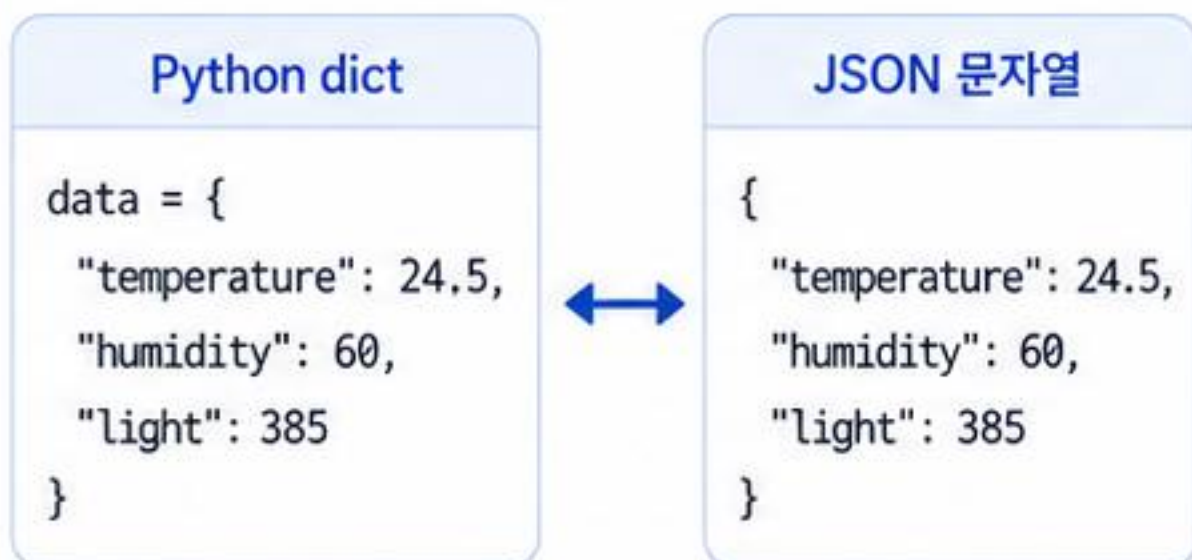
☆ 핵심 정리

- URL은 자원의 위치를 나타내고, HTTP 메서드는 그 자원에 대해 어떤 작업을 할지 정의합니다.
- REST API는 이 원칙을 기반으로, 일관된 URL과 메서드 조합으로 기능을 설계합니다.

JSON 데이터 형식

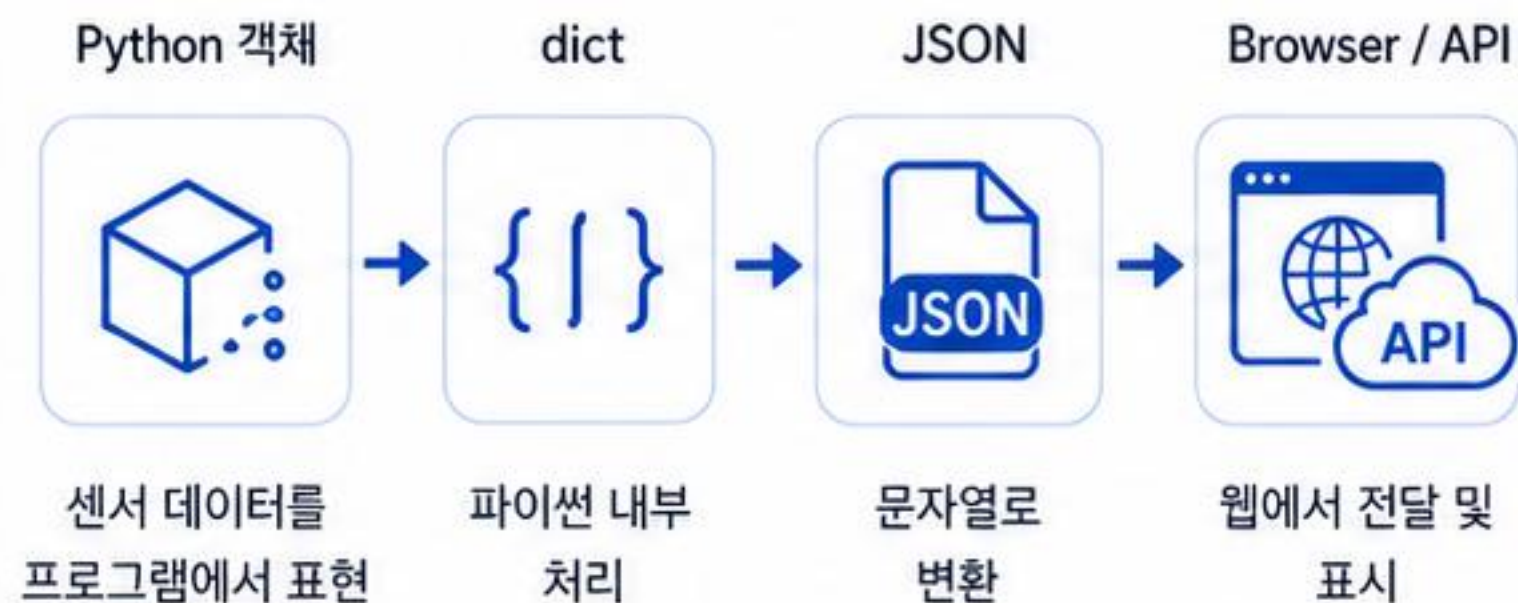
웹 API에서는 센서 데이터를 JSON 형태로 주고받습니다.

1. Python dict와 JSON 비교



- dict는 Python 내부 자료구조
- JSON은 문자열 기반 데이터 교환 형식
- 구조는 비슷하지만 사용 목적이 다름

2. 데이터 변환 흐름



3. Flask API 응답 예시

```
{
  "temperature": 24.5,
  "humidity": 60,
  "light": 385,
  "time": "2026-05-20 10:30:00"
}
```



Point

JSON은 사람이 읽기 쉽고, 프로그램도 쉽게 처리할 수 있어 웹 API에서 가장 많이 사용됩니다.



핵심 정리: Python 데이터는 dict로 다루고, 웹에서는 JSON으로 주고받습니다.

Flask란?

Python으로 웹 서버와 API를 쉽게 만들 수 있는 가벼운 웹 프레임워크



Flask는 Python으로 작성된 **마이크로 웹 프레임워크**로,
필수 기능만 제공하여 **가볍고 유연하며** 학습과 개발이 쉽습니다.

Flask의 주요 특징

1 간단한 구조



불필요한 구성 없이
최소한의 코드로 동작

2 빠른 개발



적은 코드로 빠르게
웹 서버를 구축

3 REST API 작성



클라이언트와 통신하는
REST API 구현이 용이

4 템플릿 지원



HTML 템플릿(Jinja2)
으로 동적 페이지 구성

5 IoT 서버에 적합



가볍고 리소스 효율적
으로 IoT 환경에 적합

Flask 동작 흐름

Python



애플리케이션 언어

Flask



웹 서버 / 라우팅
템플릿 / API 처리

Web / App / API



클라이언트 / 브라우저
대시보드 / 외부 시스템



핵심 포인트

Flask는 가볍고 유연한 구조로,
IoT 모니터링 서버와 **REST API**를 빠르게 구축하는 데
최적화된 웹 프레임워크입니다.



Flask 설치와 첫 서버 실행

pip install flask로 설치하고, 가장 간단한 Flask 앱을 실행해 봅시다.

1. 설치

터미널(명령 프롬프트)

```
$ pip install flask
```

설치 참고

- Flask 패키지 설치
- 가상환경(venv)에서 설치 권장
- 설치 후 import 가능

2. app.py 기본 코드

```
1 from flask import Flask
2
3 app = Flask(__name__)
4
5 @app.route("/")
6 def index():
7     return "Hello Flask"
8
9 if __name__ == "__main__":
10     app.run(debug=True)
```

3. 실행

터미널(명령 프롬프트)

```
$ python app.py
```



설치



코드 작성



서버 실행

4. 실행 결과



http://localhost:5000



Hello Flask



Flask 개발 서버 실행 중



핵심 정리

Flask는 몇 줄의 코드만으로도 웹 서버를 바로 실행할 수 있습니다.



실습 팁 (Point)

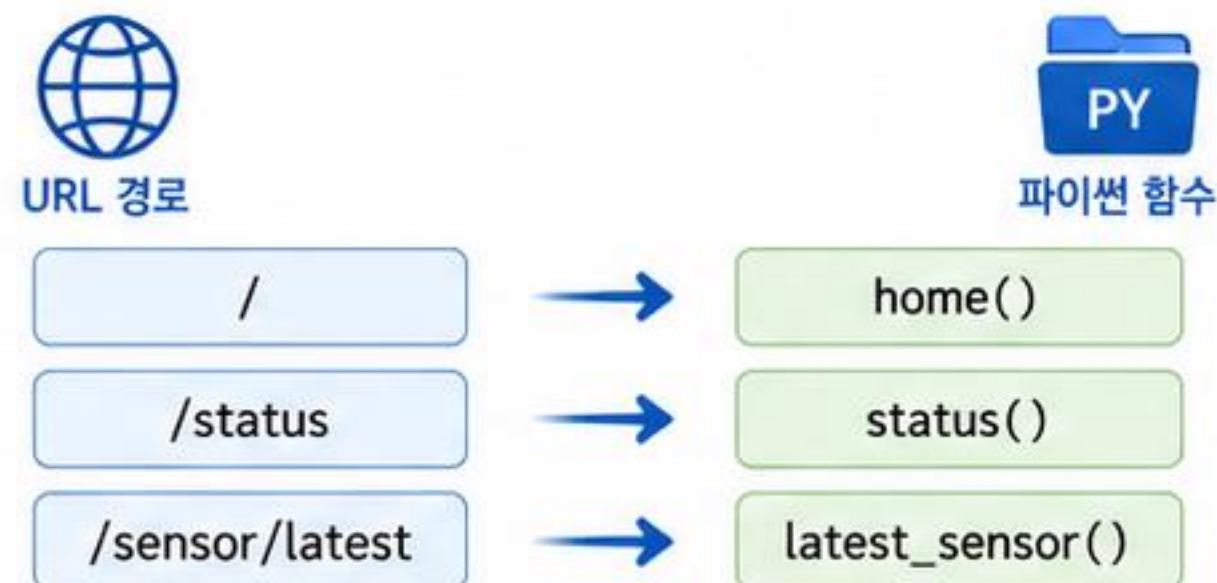
- route가 URL과 함수를 연결
- localhost:5000에서 확인
- debug=True는 개발 중에만 사용

Route와 View 함수

URL 경로와 파이썬 함수를 연결해 웹 서버의 기능을 만듭니다.

1. Route란?

- @app.route()는 URL 경로를 등록합니다
- 브라우저 요청이 들어오면 연결된 함수가 실행됩니다
- 예: /, /status, /sensor/latest



핵심 개념

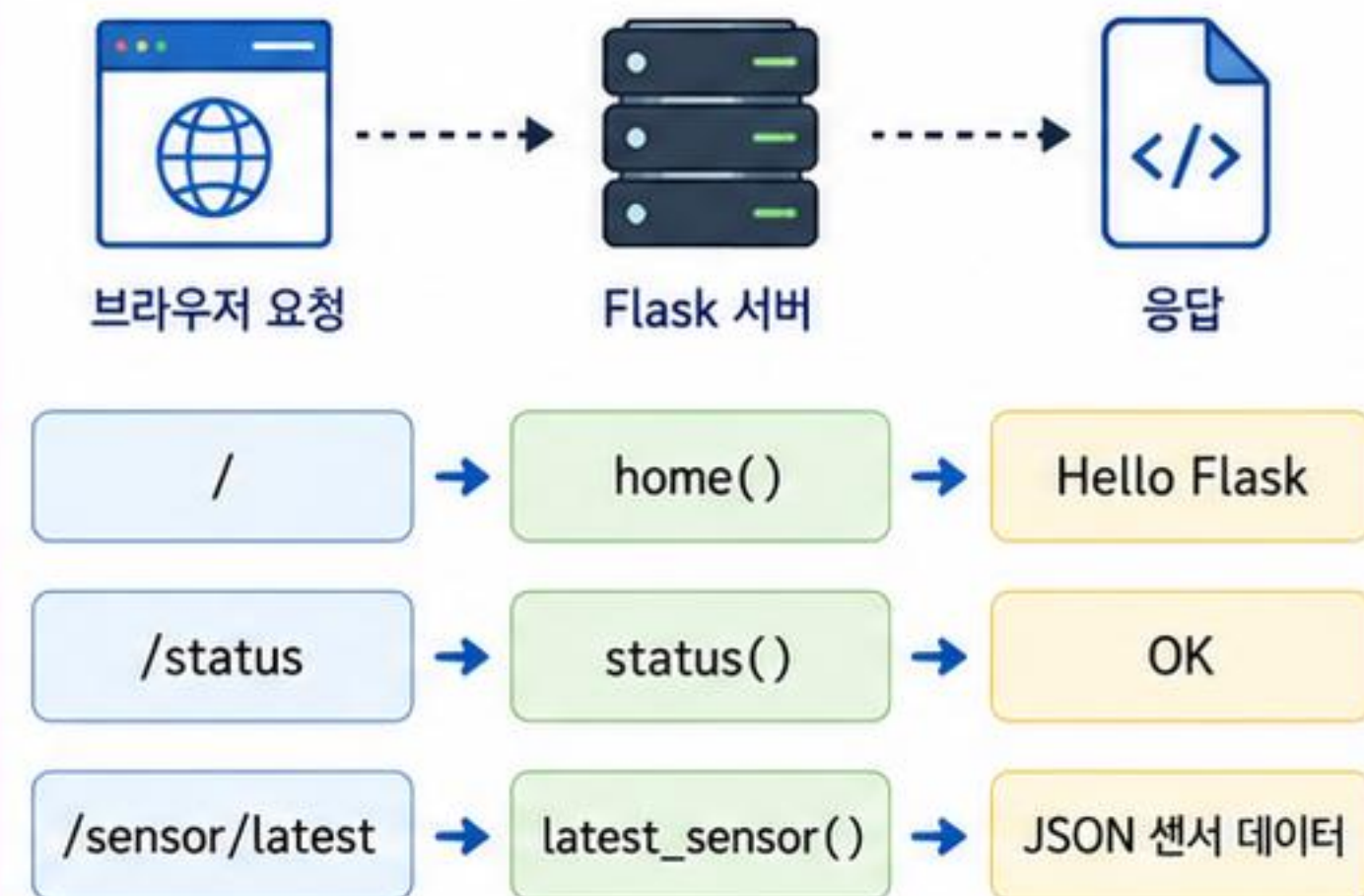


2. 코드 예시

```

1 from flask import Flask
2 app = Flask(__name__)
3
4 @app.route("/")
5 def home():
6     return "Hello Flask"
7
8 @app.route("/status")
9 def status():
10    return "OK"
11
12 @app.route("/sensor/latest")
13 def latest_sensor():
14    return {"temperature": 24.5, "humidity": 60}
15
16 if __name__ == "__main__":
17    app.run(debug=True)
18
  
```

3. URL과 함수 연결



★ 실습 팁 (Point)

- 🔗 URL 하나는 서버 기능 하나와 연결됩니다
- </> 함수 이름은 자유롭게 정할 수 있습니다
- 🚀 return 값이 브라우저 응답이 됩니다

REST API 설계하기

센서 데이터를 웹 서비스로 제공하기 위해 URL과 HTTP 메서드를 체계적으로 설계합니다.

주요 API 목록

메서드	경로	설명
GET	/api/sensors	전체 조회
GET	/api/sensors/latest	최신 데이터 조회
POST	/api/sensors	데이터 등록
GET	/api/logs	로그 조회

① 위 엔드포인트는 예시이며, 실제 서비스 요구사항에 따라 확장될 수 있습니다.

REST API 구조

클라이언트

브라우저 또는
외부 프로그램



요청
(Request)



Flask API 서버

Python Flask
애플리케이션



저장



데이터베이스

센서 데이터
저장소



응답
(JSON)



API는 다른 프로그램이 사용할 수 있는 서버의 입구입니다.



표준화된 데이터 접근
일관된 형식과 규칙으로
데이터를 안전하게 제공



외부 시스템 연동 가능
다양한 시스템과 손쉽게
연결하여 확장 가능



웹/앱/장비에서 공통 사용
브라우저, 모바일 앱, IoT 장비 등
여러 환경에서 동일하게 사용

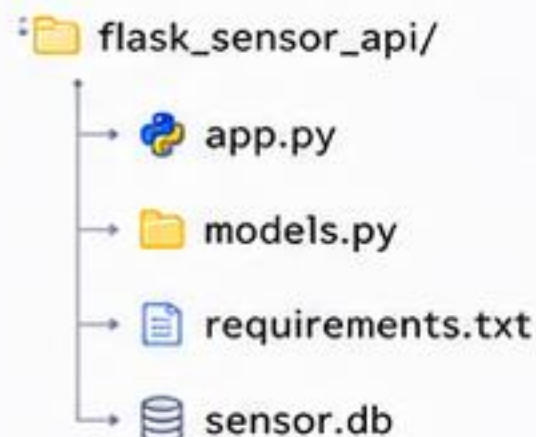
센서 데이터를 관리하는 REST API를 Flask로 구현해 보겠습니다.



Point

Flask의 라우팅 기능과 HTTP 메서드를 활용해
센서 데이터의 CRUD(Create, Read, Update, Delete)를 구현합니다.

1. 프로젝트 구조



2. 데이터 모델 (models.py)

```

from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()

class SensorData(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    temperature = db.Column(db.Float, nullable=False)
    humidity = db.Column(db.Float, nullable=False)
    created_at = db.Column(db.DateTime, server_default=db.func.now())

    def to_dict(self):
        return {
            "id": self.id,
            "temperature": self.temperature,
            "humidity": self.humidity,
            "created_at": self.created_at.isoformat()
        }
  
```

3. API 엔드포인트 및 기능

메서드	엔드포인트	설명
GET	/api/sensor	모든 센서 데이터 조회
GET	/api/sensor/<int:id>	특정 센서 데이터 조회
POST	/api/sensor	새 센서 데이터 생성
PUT	/api/sensor/<int:id>	센서 데이터 전체 수정
DELETE	/api/sensor/<int:id>	센서 데이터 삭제

4. 주요 API 코드 (app.py)

모든 센서 데이터 조회

```

@app.route('/api/sensor', methods=['GET'])
def get_all():
    data = SensorData.query.order_by(
        SensorData.created_at.desc()).all()
    return jsonify([d.to_dict() for d in data]), 200
  
```

새 센서 데이터 생성

```

@app.route('/api/sensor', methods=['POST'])
def create():
    req = request.get_json()
    new_data = SensorData(
        temperature=req['temperature'],
        humidity=req['humidity']
    )
    db.session.add(new_data)
    db.session.commit()
    return jsonify(new_data.to_dict()), 201
  
```

센서 데이터 전체 수정

```

@app.route('/api/sensor/<int:id>', methods=['PUT'])
def update(id):
    data = SensorData.query.get_or_404(id)
    req = request.get_json()
    data.temperature = req.get('temperature', data.temperature)
    data.humidity = req.get('humidity', data.humidity)
    db.session.commit()
    return jsonify(data.to_dict()), 200
  
```

센서 데이터 삭제

```

@app.route('/api/sensor/<int:id>', methods=['DELETE'])
def delete(id):
    data = SensorData.query.get_or_404(id)
    db.session.delete(data)
    db.session.commit()
    return '', 204
  
```

5. 실행 및 테스트

서버 실행

```

$ python app.py
• Running on http://127.0.0.1:5000
  
```

테스트 예시 (cURL)

```

# 전체 조회
curl http://localhost:5000/api/sensor

# 생성
curl -X POST http://localhost:5000/api/sensor \
-H "Content-Type: application/json" \
-d '{"temperature": 24.5, "humidity": 60}'

# 수정
curl -X PUT http://localhost:5000/api/sensor/1 \
-H "Content-Type: application/json" \
-d '{"temperature": 25.0}'

# 삭제
curl -X DELETE http://localhost:5000/api/sensor/1
  
```

6. 응답 예시 (GET /api/sensor)

```

[
  {
    "id": 1,
    "temperature": 24.5,
    "humidity": 60.0,
    "created_at": "2024-06-01T10:00:00"
  },
  {
    "id": 2,
    "temperature": 25.1,
    "humidity": 58.0,
    "created_at": "2024-06-01T10:05:00"
  }
]
  
```

✓ 정상 응답
HTTP Status: 200 OK



핵심 정리

- Flask의 라우팅과 HTTP 메서드를 활용해 센서 데이터의 CRUD API를 구현할 수 있습니다.
- SQLAlchemy 모델과 JSON 응답을 사용해 데이터를 안전하게 관리하고 제공합니다.
- RESTful API 설계 원칙에 맞게 자원(센서 데이터)을 중심으로 URL과 메서드를 구성합니다.

구현한 Flask REST API가 정상적으로 동작하는지 다양한 방법으로 테스트해 보겠습니다.



Point

API 테스트는 개발 단계에서 오류를 빠르게 발견하고, 클라이언트(IoT 디바이스)와의 연동을 검증하는 데 필수적입니다.

1. 테스트 준비

✓ 서버 실행 확인

```
$ python app.py
Running on http://127.0.0.1:5000
```

✓ API 엔드포인트 확인

/api/sensor (GET, POST)

✓ 테스트 도구

- 웹 브라우저
- Postman 또는 Insomnia
- curl 명령어

2. 웹 브라우저로 테스트 (GET)

URL 입력

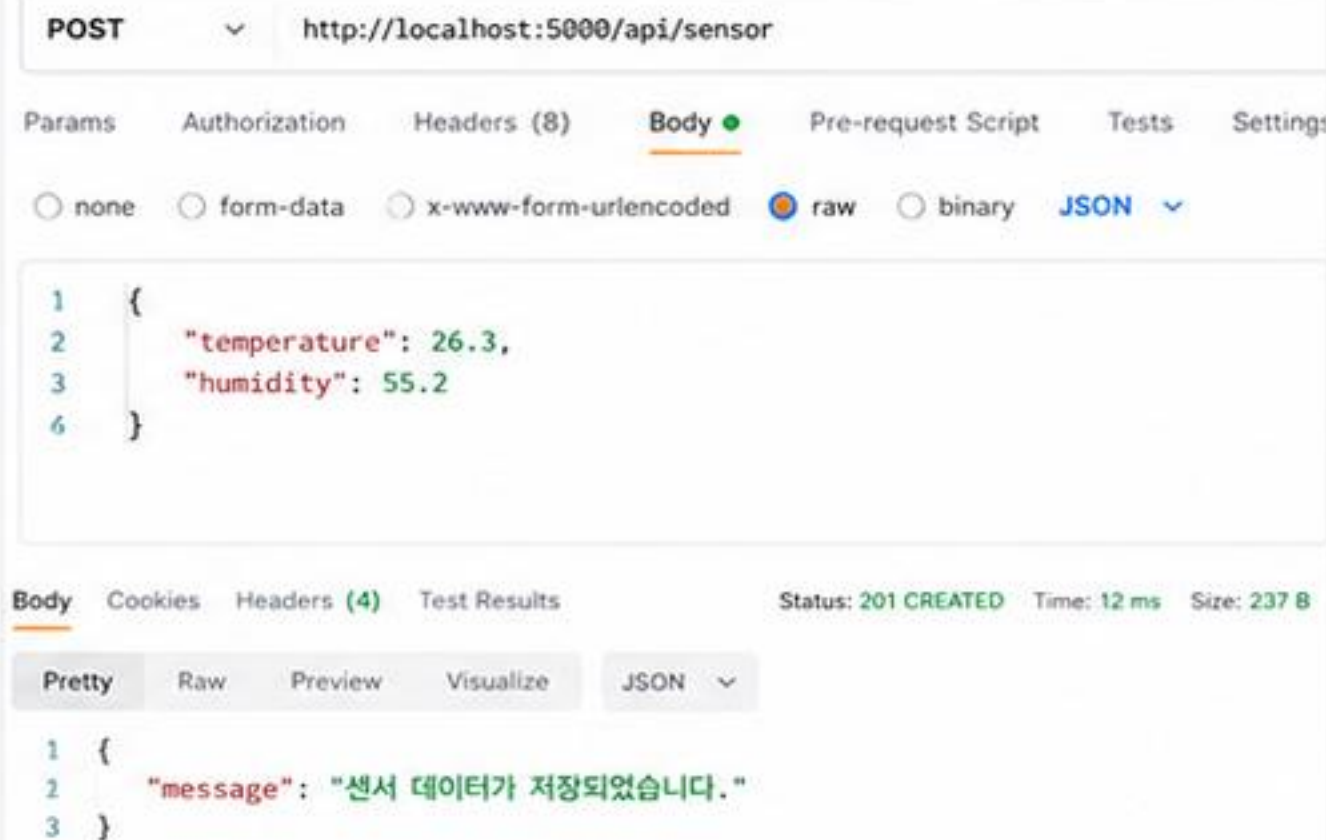
http://localhost:5000/api/sensor

결과 (JSON 응답)

```
[
  {
    "id": 1,
    "temperature": 24.5,
    "humidity": 60.0,
    "created_at": "2024-06-01T10:00:00"
  },
  {
    "id": 2,
    "temperature": 25.1,
    "humidity": 58.0,
    "created_at": "2024-06-01T10:05:00"
  }
]
```

✓ 정상 응답: 200 OK

3. Postman으로 테스트



4. curl 명령어로 테스트

GET 요청 (모든 데이터 조회)

```
curl http://localhost:5000/api/sensor
```

특정 ID 조회

```
curl http://localhost:5000/api/sensor/1
```

POST 요청 (새 데이터 생성)

```
curl -X POST http://localhost:5000/api/sensor \
-H "Content-Type: application/json" \
-d '{"temperature":26.8,"humidity":54.3}'
```

PUT 요청 (데이터 수정)

```
curl -X PUT http://localhost:5000/api/sensor/1 \
-H "Content-Type: application/json" \
-d '{"temperature":27.0,"humidity":53.0}'
```

DELETE 요청 (데이터 삭제)

```
curl -X DELETE http://localhost:5000/api/sensor/1
```

5. 테스트 시나리오

시나리오	메서드	URL	설명	예상 결과
모든 데이터 조회	GET	/api/sensor	모든 센서 데이터 조회	200 OK + JSON 배열
특정 데이터 조회	GET	/api/sensor/<id>	특정 ID의 데이터 조회	200 OK + JSON 객체
새 데이터 생성	POST	/api/sensor	새 센서 데이터 저장	201 Created + 메시지
데이터 수정	PUT	/api/sensor/<id>	기존 데이터 수정	200 OK + 메시지
데이터 삭제	DELETE	/api/sensor/<id>	데이터 삭제	200 OK + 메시지
존재하지 않는 ID 조회	GET	/api/sensor/999	없는 ID 조회	404 Not Found
잘못된 데이터 전송	POST	/api/sensor	필수 필드 누락	400 Bad Request

6. 테스트 체크리스트

- ✓ 모든 API 엔드포인트가 정상적으로 응답하는가?
- ✓ GET, POST, PUT, DELETE가 올바르게 동작하는가?
- ✓ 잘못된 요청에 대해 적절한 에러 코드가 반환되는가?
- ✓ 응답 데이터가 JSON 형식으로 올바르게 반환되는가?
- ✓ 데이터베이스에 데이터가 정상적으로 저장/수정/삭제되는가?

7. 문제 해결 팁

❗ 연결 오류	→ 서버가 실행 중인지 확인 (python app.py)
❗ 404 오류	→ URL 경로가 올바른지 확인
❗ 400 오류	→ 요청 데이터 형식(JSON) 확인
❗ 500 오류	→ 서버 로그 확인 및 코드 오류 점검
❗ CORS 오류	→ 필요시 CORS 설정 추가



핵심 정리

- 다양한 도구(브라우저, Postman, curl)를 사용해 API를 테스트할 수 있습니다.
- CRUD 기능이 모두 정상 동작하는지 확인해야 합니다.
- 에러 케이스 테스트도 중요합니다.
- 테스트는 안정적인 API 개발의 필수 과정입니다.



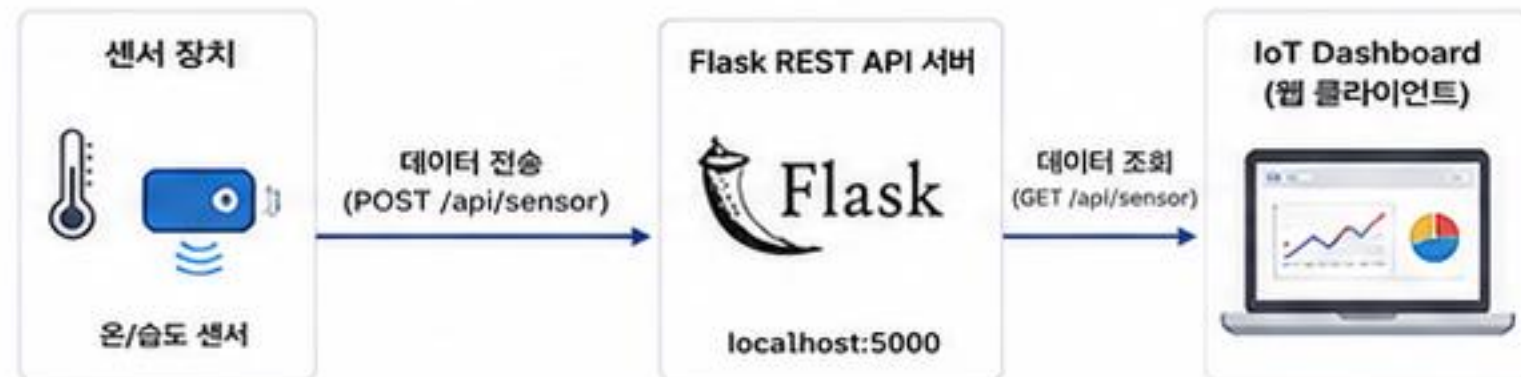
구현한 REST API를 IoT Dashboard(웹 클라이언트)와 연동하여 실시간 센서 데이터를 확인해 보겠습니다.



Point

대시보드에서 주기적으로 API를 호출하여
센서 데이터를 가져오고, 시각화하여 표시합니다.

1. 연동 구조



- 센서 → 서버: 실시간 데이터 전송 (POST)
- 대시보드 → 서버: 데이터 조회 (GET)
- 서버 → 대시보드: JSON 데이터 응답

4. 대시보드 JavaScript (계속)

```

async function fetchData() {
  try {
    const response = await fetch(API_URL);
    if (!response.ok) throw new Error('API 오류');
    const data = await response.json();

    // 상태 업데이트
    document.getElementById('status').textContent = '연결 상태: 정상'
    document.getElementById('last-update').textContent =
      '마지막 업데이트: ' + new Date(data.timestamp).toLocaleTimeString();

    // 값 업데이트
    document.getElementById('temperature').textContent = data.temperature.toFixed(1);
    document.getElementById('humidity').textContent = data.humidity.toFixed(1);

    // 차트 데이터 업데이트
    const time = new Date(data.timestamp).toLocaleTimeString();
    if (labels.length >= 20) { labels.shift(); tempData.shift(); humData.shift(); }
    labels.push(time);
    tempData.push(data.temperature);
    humData.push(data.humidity);
    chart.update();
  } catch (error) {
    document.getElementById('status').textContent = '연결 상태: 오류';
    console.error(error);
  }
}
  
```

2. 대시보드 HTML (dashboard/index.html)

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>IoT Dashboard</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>🌟 IoT 센서 대시보드</h1>
  <div class="status">
    <span id="status">연결 상태: 확인 중...</span>
    <span id="last-update"></span>
  </div>

  <div class="cards">
    <div class="card">
      <h3>온도 (°C)</h3>
      <div id="temperature" class="value">--</div>
    </div>
    <div class="card">
      <h3>습도 (%)</h3>
      <div id="humidity" class="value">--</div>
    </div>
  </div>

  <div class="chart-container">
    <canvas id="chart"></canvas>
  </div>

  <script src="script.js"></script>
</body>
</html>
  
```

5. CSS (dashboard/style.css)

```

body { font-family: 'Segoe UI', sans-serif; margin: 20px;
background: #f4f6f8; }
h1 { color: #333; }
status { margin-bottom: 20px; font-size: 14px; }
cards { display: flex; gap: 20px; margin-bottom: 20px; }
card {
  background: white; padding: 20px; border-radius: 8px;
  box-shadow: 0 2px 4px rgba(0,0,0,.1); flex: 1; text-align: center; }
card h3 { margin: 0 0 10px 0; color: #666; }
value { font-size: 32px; font-weight: bold; color: #2196f3; }
chart-container { background: white; padding: 20px; border-radius: 8px;
box-shadow: 0 2px 4px rgba(0,0,0,.1); }
canvas { max-height: 300px; }
  
```

3. 대시보드 JavaScript (dashboard/script.js)

```

const API_URL = 'http://localhost:5000/api/sensor';
let chart;
const labels = [];
const tempData = [];
const humData = [];

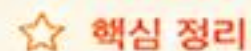
// 초기화
window.onload = function() {
  initChart();
  fetchData();
  setInterval(fetchData, 5000); // 5초마다 갱신
};

function initChart() {
  const ctx = document.getElementById('chart').getContext('2d');
  chart = new Chart(ctx, {
    type: 'line',
    data: {
      labels: labels,
      datasets: [
        {
          label: '온도 (°C)',
          data: tempData,
          borderColor: 'rgba(255, 99, 132, 1)',
          tension: 0.4, fill: false
        },
        {
          label: '습도 (%)',
          data: humData,
          borderColor: 'rgba(54, 162, 235, 1)',
          tension: 0.4, fill: false
        }
      ]
    },
    options: {
      responsive: true,
      scales: {
        y: { beginAtZero: false }
      }
    }
  });
}
  
```

... (다음 페이지 계속)

6. 실행 방법

- 1 Flask 서버 실행 (app.py)
\$ python app.py
- 2 대시보드 파일 열기
index.html을 웹 브라우저에서 열기
- 3 센서 데이터 전송
다른 클라이언트(또는 대시보드) 테스트 버튼으로 POST 요청 전송
- 4 데이터 확인
대시보드에서 실시간으로 온/습도 값과 그래프 확인



핵심 정리

- Flask REST API가 센서 데이터를 제공하고, 웹 대시보드가 이를 시각화합니다.
- fetch() API를 사용해 주기적으로 데이터를 조회합니다.
- Chart.js를 활용해 온도/습도 변화를 그래프로 표시합니다.
- IoT 시스템의 전체 흐름을 완성했습니다!



완성한 Flask REST API를 외부에서 접근할 수 있도록 배포해 보겠습니다.



Point

배포는 IoT 서비스의 핵심 단계입니다.

이번 단계에서는 Render를 이용해 간단히 클라우드 배포를 진행합니다.

1. 배포 준비

- 필요 파일 확인
 - app.py
 - requirements.txt
 - models.py
 - sensor.db (선택: 초기 데이터)

requirements.txt 예시

```
Flask==3.0.3
Flask-CORS==4.0.1
SQLAlchemy==2.0.30
gunicorn==22.0.0
```

2. Render 회원가입 및 새 Web Service 생성

- ① <https://render.com> 회원가입/로그인
- ② New + → Web Service 선택
- ③ GitHub 저장소 연결 (프로젝트 선택)
- ④ 다음과 같이 설정 후 Create Web Service 클릭

Name	flask-iot-api
Environment	Python 3
Build Command	pip install -r requirements.txt
Start Command	gunicorn app:app
Plan	Free

Create a new Web Service

Connect a repository
your-username / flask-iot-api

Name
flask-iot-api

Environment
Python 3

Build Command
pip install -r requirements.txt

Start Command
gunicorn app:app

Create Web Service

3. 배포 진행 및 확인

- ① 배포가 시작되면 빌드 로그 확인
- ② 배포가 완료되면 서비스 URL 확인
- ③ URL 예시
`https://flask-iot-api-xxxx.onrender.com`
- ④ /api/sensor 엔드포인트 테스트

```
curl https://flask-iot-api-xxxx.onrender.com/api/sensor
```

정상 응답 예시

```
{
  {
    "id": 1,
    "temperature": 24.8,
    "humidity": 60.5,
    "created_at": "2024-06-01T10:00:00"
  }
}
```

4. 환경 변수 설정 (선택 사항)

- ① Settings → Environment 탭 이동
- ② 필요 시 환경 변수 추가

Key	Value 예시
FLASK_ENV	production
DATABASE_URL	sqlite:///sensor.db
SECRET_KEY	your-secret-key

5. 무료 플랜 제한사항

- 서비스가 15분 이상 비활성 상태이면 슬립 모드로 전환
- 다음 요청이 오면 다시 시작 (수초 소요)
- 데이터는 지속되지만, 응답 지연 가능

운영 환경에서는 유료 플랜 또는 외부 DB(PostgreSQL 등) 사용 권장

6. 배포 후 구조

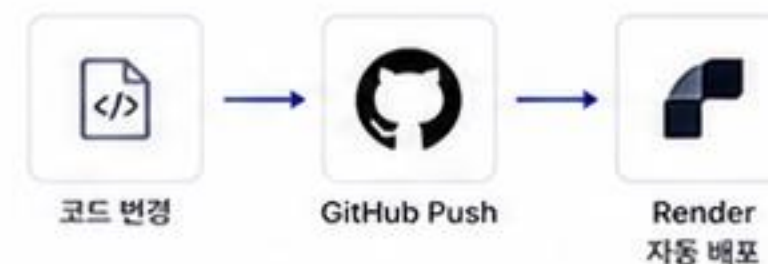


7. 배포 후 확인 체크리스트

- /api/sensor (GET) 정상 응답
- /api/sensor (POST) 데이터 등록
- /api/sensor/<id> (PUT) 데이터 수정
- /api/sensor/<id> (DELETE) 데이터 삭제

8. 자동 재배포

GitHub 저장소에 코드 푸시 시 자동으로 재배포됩니다.



☆ 핵심 정리

- Flask 앱을 클라우드에 배포하여 언제 어디서나 접근 가능한 IoT API를 완성했습니다.
- Render를 사용하면 무료로 간편하게 배포할 수 있습니다.
- 배포된 API URL을 IoT 디바이스나 대시보드에서 활용할 수 있습니다.
- 다음 단계: IoT 디바이스에서 실제 API 연동 테스트!

Day 4에서 구현한 IoT 센서 REST API 실습 내용을 한눈에 정리합니다.



핵심 Point

- 센서 데이터를 **CRUD** 로 관리하는 REST API를 **Flask**로 구현
- 다양한 도구와 방법으로 API를 테스트하고 활용

1. 구현한 API 기능 (CRUD)

GET	/api/sensor	모든 센서 데이터 조회
GET	/api/sensor/<id>	특정 센서 데이터 조회
POST	/api/sensor	새 센서 데이터 생성
PUT	/api/sensor/<id>	센서 데이터 전체 수정
DELETE	/api/sensor/<id>	센서 데이터 삭제

2. 데이터 모델 (SQLAlchemy)

sensor	
id	Integer (PK)
temperature	Float
humidity	Float
created_at	DateTime

↔  SQLite

3. API 테스트 도구

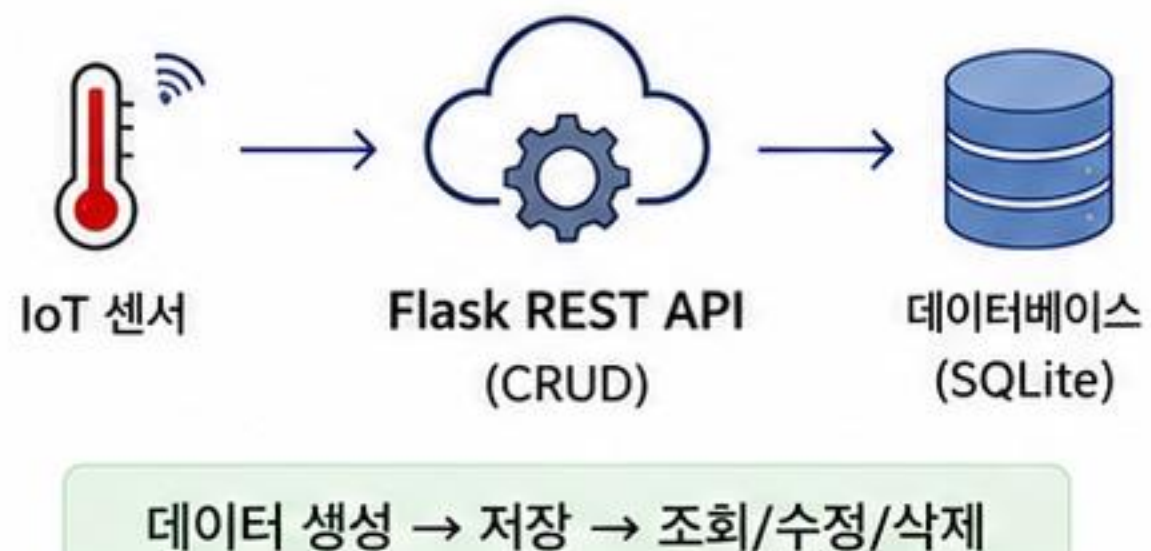
	Postman	API 요청/응답 테스트
	curl	명령어로 API 테스트
	웹 브라우저	GET 요청 결과 확인

4. 실행 및 환경



- ✓ Flask 웹 서버 실행
python app.py
- ✓ 로컬 서버 주소
http://127.0.0.1:5000
- ✓ DB: SQLite (sensor.db)

5. API 활용 흐름



6. 배운 내용 핵심 정리

- ✓ Flask로 REST API 서버 구축
- ✓ SQLAlchemy로 데이터 모델링 및 CRUD 구현
- ✓ JSON 형식으로 데이터 주고받기
- ✓ Postman, curl 등으로 API 테스트
- ✓ IoT 센서 데이터를 안전하게 관리

다양한 기능을 추가하여 IoT 센서 데이터를 더 효율적으로 관리해 보겠습니다.



핵심 Point

- 필터링, 정렬, 페이지네이션을 적용하여 데이터를 효율적으로 조회
- 에러 처리 및 데이터 검증으로 안정적인 API 구현

1. 고급 기능 추가



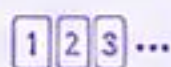
필터링

특정 조건(온도 범위, 날짜 등)으로 데이터 조회



정렬

특정 필드(temperature 등) 기준 정렬



페이지네이션

대량 데이터 조회 시 페이지 단위로 분할



데이터 검증 & 에러 처리

잘못된 요청에 대한 적절한 응답 제공

2. 필터링 및 정렬 예 코드

```
@app.route('/api/sensor')
def get_sensor():
    min_temp = request.args.get('min_temp', type=float)
    max_temp = request.args.get('max_temp', type=float)
    order_by = request.args.get('order_by', 'created_at')
    order = request.args.get('order', 'desc')

    query = SensorData.query
    if min_temp is not None:
        query = query.filter(SensorData.temperature >= min_temp)
    if max_temp is not None:
        query = query.filter(SensorData.temperature <= max_temp)
    if order == 'asc':
        query = query.order_by(getattr(SensorData, order_by).asc())
    else:
        query = query.order_by(getattr(SensorData, order_by).desc())
    return jsonify([item.to_dict() for item in query.all()])
```

쿼리 파라미터로
필터링, 정렬
적용

3. 페이지네이션 예시 코드

```
@app.route('/api/sensor')
def get_sensor():
    page = request.args.get('page', 1, type=int)
    per_page = request.args.get('per_page', 10, type=int)

    pagination = SensorData.query.order_by(
        SensorData.created_at.desc()
    ).paginate(page=page, per_page=per_page, error_out=False)

    return jsonify({
        'data': [item.to_dict() for item in pagination.items],
        'page': pagination.page,
        'per_page': pagination.per_page,
        'total_pages': pagination.pages,
        'total_count': pagination.total
    })
```

페이지 단위로
데이터 조회
및 메타 정보 제공

4. API 테스트 예시

GET	/api/sensor	전체 데이터 조회
GET	/api/sensor?min_temp=20&max_temp=30	온도 20~30도 데이터 조회
GET	/api/sensor?order_by=temperature&order=asc	온도 기준 오름차순 정렬
GET	/api/sensor?page=2&per_page=5	2페이지, 5개씩 조회
POST	/api/sensor	새 센서 데이터 등록

응답 예시

```
{
  "data": [
    { "id": 1, "temperature": 24.5, ... }
  ],
  "page": 2,
  "per_page": 5,
  "total_pages": 10,
  "total_count": 48
}
```

5. 다음 단계



데이터 시각화

Chart.js 등을 활용한
대시보드 구축



알림 기능

특정 임계값 초과 시
알림(Email, SMS 등)



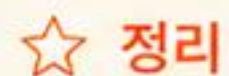
클라우드 연동

AWS, Google Cloud 등
클라우드 서비스 연동



보안 강화

API Key, JWT 인증 등으로
보안 강화



정리

필터링, 정렬, 페이지네이션 등 다양한 기능을 추가하여 API를 더 강력하게 만들었습니다.

안정적인 API 설계를 통해 실제 서비스에 적용할 수 있는 수준으로 발전시켜 나가세요!

18 Flask REST API 프로젝트 실습

지금까지 배운 내용을 활용하여 IoT 센서 데이터를 관리하는 REST API 프로젝트를 만들어 봅시다.



핵심 Point

- 전체 흐름을 이해하고 직접 구현해보며 실력을 완성
- CRUD 기능과 다양한 API를 포함한 완성도 높은 프로젝트

1 프로젝트 목표



IoT 센서 데이터를 수집, 저장, 조회, 관리하는 REST API 서버 구축

구현 기능

- ✓ 센서 데이터 CRUD
- ✓ 필터링, 정렬, 페이지네이션
- ✓ 통계 및 대시보드 API
- ✓ 에러 처리 및 검증

2 기술 스택



Python 3



Flask



SQLAlchemy



SQLite

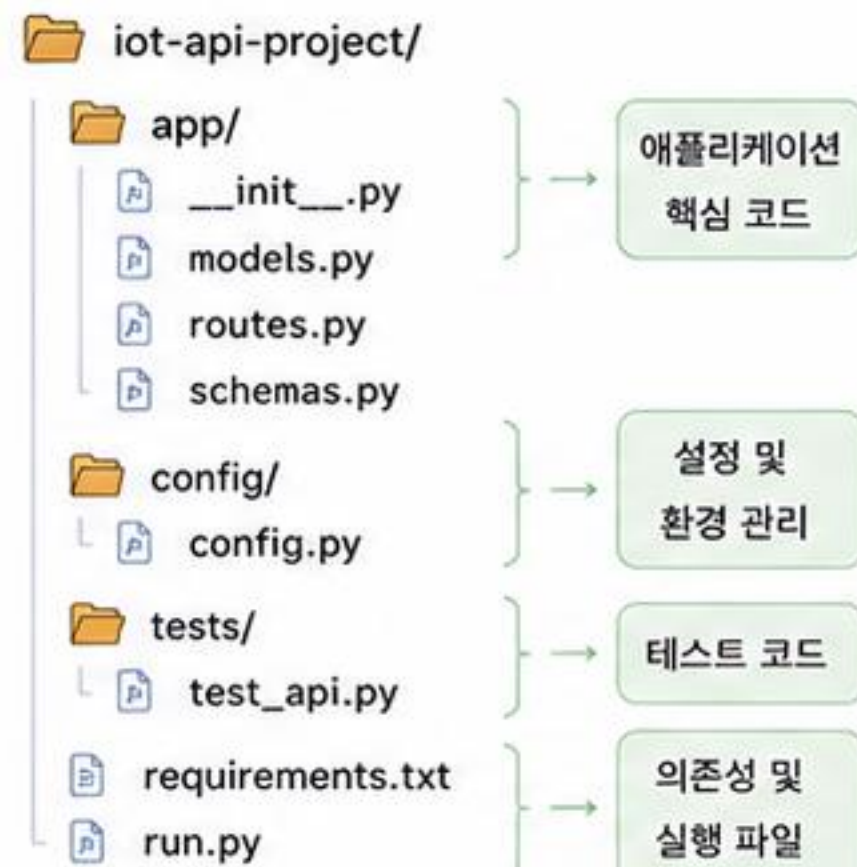


Postman

3 주요 엔드포인트

GET	/api/sensor	전체 데이터 조회
GET	/api/sensor/<id>	단일 데이터 조회
POST	/api/sensor	새 데이터 생성
PUT	/api/sensor/<id>	데이터 전체 수정
DELETE	/api/sensor/<id>	데이터 삭제
GET	/api/stats	통계 정보 조회
GET	/api/dashboard	대시보드 요약 정보

4 프로젝트 구조



5 개발 단계



6 시스템 흐름도



7 완성 후 테스트 항목

- ✓ 모든 CRUD 기능 정상 동작 확인
- ✓ 필터링, 정렬, 페이지네이션 동작 확인
- ✓ 통계 및 대시보드 API 응답 확인
- ✓ 에러 처리 및 예외 상황 테스트
- ✓ Postman으로 전체 API 테스트

8 확장 아이디어

- ☆ 사용자 인증(JWT) 추가
- ☆ 센서 데이터 실시간 알림
- ☆ 차트 기반 대시보드 연동
- ☆ 데이터 분석 기능 추가
- ☆ 클라우드 배포 (AWS, GCP)



정리

이제 여러분은 Flask와 REST API를 활용하여 실제 IoT 서비스를 만들 수 있습니다!



꾸준한 실습과 프로젝트 경험이 최고의 실력을 만듭니다.



도전하고 구현하며 성장하세요!



지금까지 배운 Flask REST API를 활용하여 IoT 센서 데이터를 시각화하는 대시보드 웹 애플리케이션을 만들어 봅시다.



핵심 Point

- REST API로 센서 데이터를 제공하고
- 대시보드에서 실시간으로 시각화합니다.

1. 프로젝트 목표



- ✓ 센서 데이터를 REST API로 제공
- ✓ 대시보드에서 데이터 조회 및 시각화
- ✓ 실시간 데이터 갱신 및 기록 관리

2. 기술 스택



3. 주요 기능

- ✓ **센서 데이터 CRUD**
생성, 조회, 수정, 삭제
- ✓ **데이터 API 제공**
REST API로 JSON 데이터 반환
- ✓ **대시보드 시각화**
온도, 습도 그래프 및 카드 표시
- ✓ **데이터 필터링 및 정렬**
기간별 조회 및 정렬 기능
- ✓ **자동 갱신**
주기적으로 데이터 업데이트

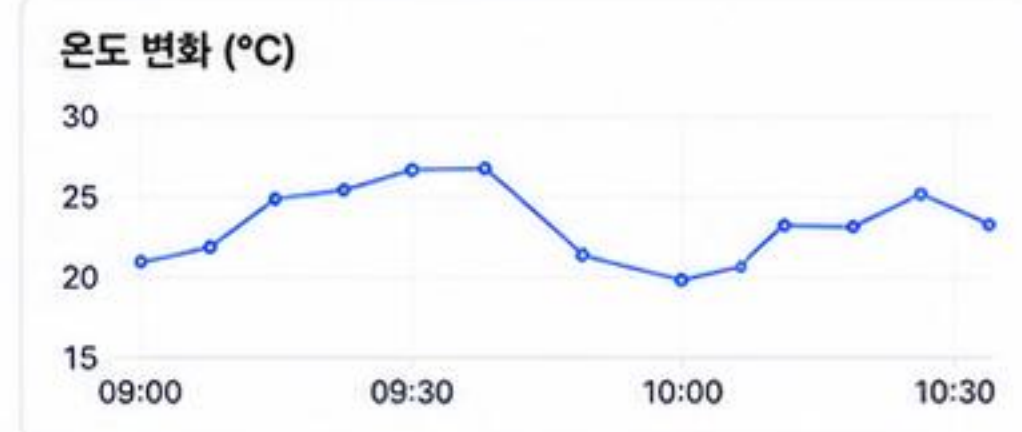
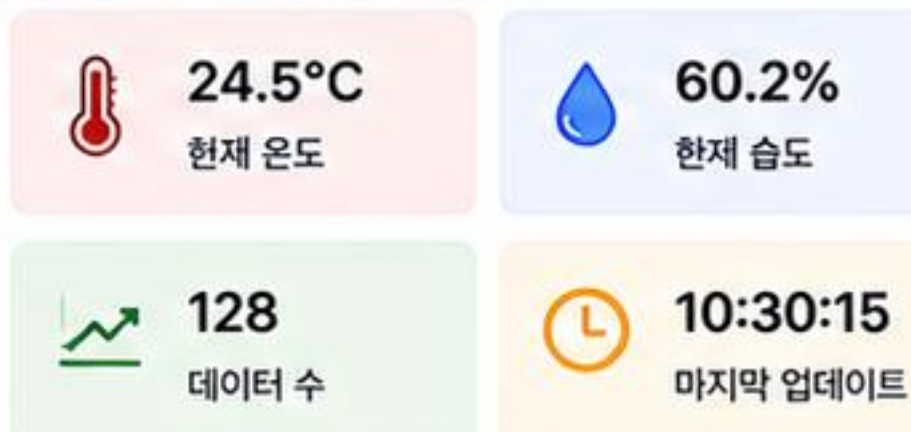
4. API 엔드포인트 예시

GET	/api/sensors	모든 센서 데이터 조회
GET	/api/sensors/<id>	특정 센서 데이터 조회
POST	/api/sensors	새 센서 데이터 생성
PUT	/api/sensors/<id>	센서 데이터 전체 수정
DELETE	/api/sensors/<id>	센서 데이터 삭제
GET	/api/dashboard	대시보드 요약 데이터

5. 데이터 흐름



6. 대시보드 예시 화면



정리

Flask REST API로 IoT 센서 데이터를 제공하고, 대시보드에서 실시간으로 시각화하는 완전한 웹 애플리케이션을 구축합니다.



실습을 통해 IoT 데이터 관리 및 웹 개발 역량을 한 단계 업그레이드하세요!

20 Day 4 최종 정리 & 마무리

Flask REST API를 활용한 IoT 센서 데이터 수집 및 대시보드 시각화 프로젝트를 완성했습니다!



핵심 Point

- IoT 센서 데이터를 REST API로 관리
- CRUD 기능으로 데이터를 안전하게 처리
- 대시보드에서 실시간으로 데이터를 시각화

1 프로젝트 개요



IoT 센서의 데이터를 수집하고, REST API로 관리하여 대시보드에서 실시간으로 시각화하는 시스템을 구축했습니다.

2 구현 기능

- ✓ 센서 데이터 CRUD (GET, POST, PUT, DELETE)
- ✓ 필터링, 정렬, 페이지네이션
- ✓ 데이터 검증 및 에러 처리
- ✓ 통계 및 대시보드 API
- ✓ 대시보드 시각화 (차트, 카드, 실시간 갱신)

3 기술 스택



4 API 엔드포인트 요약

GET	/api/sensors	모든 센서 조회
GET	/api/sensors/<id>	특정 센서 조회
POST	/api/sensors	새 센서 생성
PUT	/api/sensors/<id>	센서 데이터 수정
DELETE	/api/sensors/<id>	센서 데이터 삭제
GET	/api/dashboard	대시보드 요약 데이터

5 프로젝트 결과



- ✓ 센서 데이터의 안정적인 수집 및 관리
- ✓ REST API를 통한 효율적인 데이터 처리
- ✓ 직관적인 대시보드로 실시간 모니터링
- ✓ IoT 서비스의 핵심 기능 구현 완료!

6 배운 점

- </> Flask로 REST API 서버를 구축하고 CRUD 기능을 구현하는 방법을 익혔습니다.
- SQLAlchemy를 사용하여 데이터 모델링과 DB 관리를 경험했습니다.
- 필터링, 정렬, 페이지네이션으로 대용량 데이터를 효율적으로 처리하는 방법을 배웠습니다.
- Chart.js를 활용하여 데이터를 시각화하고 대시보드를 구현했습니다.
- IoT 센서 데이터를 실시간으로 수집하고 모니터링하는 전체 흐름을 이해했습니다.

7 다음 단계



수고하셨습니다!



Flask REST API와 IoT를 활용한 멋진 프로젝트를 완성했습니다!

이제 여러분은 IoT 서비스를 개발할 수 있는 강력한 기반을 갖추었습니다.

